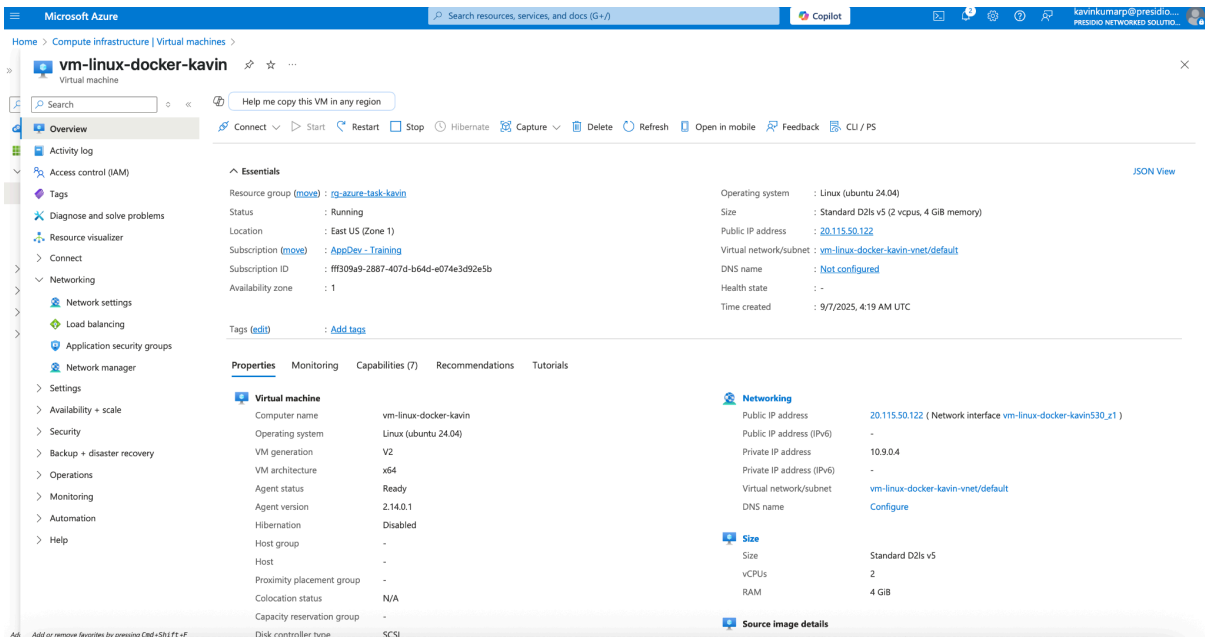


1. Created a Linux Vm in the Azure portal with a public static ip



2. Connected to the Linux Vm through SSH with a pem key

```
mv ~/Downloads/vm-linux-docker-kavin_key.pem ~/.ssh/  
chmod 400 ~/.ssh/vm-linux-docker-kavin_key.pem  
(Moved the pem key file to the ssh hidden folder)
```

ssh -i ~/.ssh/vm-linux-docker-kavin_key.pem azureuser@<public-ip> (connection)

```
kavin@vm-linux-docker-kavin:~$
```

3. Install Docker

- sudo apt-get update
- sudo apt-get install -y [docker.io](https://docs.docker.com/engine/install/debian/)
- sudo systemctl start docker
- sudo systemctl enable docker
- docker --version

```
presidio — kavin@vm-linux-docker-kavin: ~ — ssh -i ~/.ssh/vm-linux-doc...
Setting up docker.io (27.5.1-0ubuntu3~24.04.2) ...
info: Selecting GID from range 100 to 999 ...
info: Adding group `docker' (GID 114) ...
Created symlink /etc/systemd/system/multi-user.target.wants/docker.service → /usr/lib/systemd/system/docker.service.
Created symlink /etc/systemd/system/sockets.target.wants/docker.socket → /usr/lib/systemd/system/docker.socket.
Processing triggers for dbus (1.14.10-4ubuntu4.1) ...
Processing triggers for man-db (2.12.0-4build2) ...
Scanning processes...
Scanning linux images...

Running kernel seems to be up-to-date.

No services need to be restarted.

No containers need to be restarted.

No user sessions are running outdated binaries.

No VM guests are running outdated hypervisor (qemu) binaries on this host.
kavin@vm-linux-docker-kavin:~$ docker --version
Docker version 27.5.1, build 27.5.1-0ubuntu3~24.04.2
kavin@vm-linux-docker-kavin:~$
```

4. Created a Sample new webapi project in local , also create a docker file and test locally

- Possible ways to run image in the linux vm
 - docker hub pull
 - run image
 - git clone <url>
 - build and run image

```
kavin@vm-linux-docker-kavin:~/app$ git clone https://github.com/kavinkumar24/linux-vm-api.git
Cloning into 'linux-vm-api'...
remote: Enumerating objects: 52, done.
remote: Counting objects: 100% (52/52), done.
remote: Compressing objects: 100% (42/42), done.
remote: Total 52 (delta 3), reused 52 (delta 3), pack-reused 0 (from 0)
Receiving objects: 100% (52/52), 844.56 KiB | 21.11 MiB/s, done.
Resolving deltas: 100% (3/3), done.
kavin@vm-linux-docker-kavin:~/app$ cd linux-vm-api/
kavin@vm-linux-docker-kavin:~/app/linux-vm-api$ cd Myapp/
```

Edit the [program.cs](#) file to allow the all ips for binding

-> nano [program.cs](#)

- add - **builder.WebHost.Useurls("http://0.0.0.0:8080")**

(Before builder.Build())

-> save and build the image (sudo docker build -t app-dotnet-api .)

- run the container (sudo docker run -d -p 5001:8080 app-dotnet-api)

```
kavin@vm-linux-docker-kavin:~/app/linux-vm-api/Myapp$ sudo docker run -d -p 5001:8080 app-dotnet-api
09cfaa07e12c02e98c7f35fbaf01dc658369c041c127277edd1c0a7af43d229b
```

5. Check running Container

- docker ps

```
'app/linux-vm-api/Myapp$ sudo docker ps
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS
39cfaa07e12c	app-dotnet-api	"dotnet Myapp.dll"	2 hours ago	Up 2 hours	80/tcp, 0.0.0.0:5001->8080/tcp, [::]:5001->8080/tcp

```
_hugle
```

6. Check the backend url in the localsystem

The screenshot shows the Swagger UI for 'Myapp v1' (OAS 3.0) at the URL `http://20.115.50.122:5001/swagger/index.html`. The selected definition is 'Myapp v1'. The endpoint `GET /weatherforecast` is highlighted. The 'Parameters' section shows 'No parameters'. The 'Responses' section shows a '200' status code with a 'Response body' containing a JSON array of weather forecast data.

Execute **Clear**

Responses

Code **Details**

200

Response body

```
[{"date": "2025-07-10", "temperatureC": 51, "summary": "Bracing", "temperatureF": 123}, {"date": "2025-07-11", "temperatureC": -7, "summary": "Warm", "temperatureF": 20}, {"date": "2025-07-12", "temperatureC": 0, "summary": "Chilly", "temperatureF": 46}, {"date": "2025-07-13", "temperatureC": -14, "summary": "Mild", "temperatureF": 7}, {"date": "2025-07-14", "temperatureC": -5, "summary": "Hot", "temperatureF": 24}]
```

SSH tunnelling locally

Installing package in the linux vm

- `sudo apt install xfce4 -y`
- `sudo apt install xrdp -y`
- `echo "xfce4-session" > ~/.xsession`
- `sudo systemctl enable xrdp`
- `sudo systemctl start xrdp`

In the mac terminal

- `ssh -i ~/.ssh/vm-linux-docker-kavin_key.pem -L 3389:localhost:3389 kavin@20.115.50.122`
- Set a password for the username (after connecting)
 - `Sudo passwd <username>`

